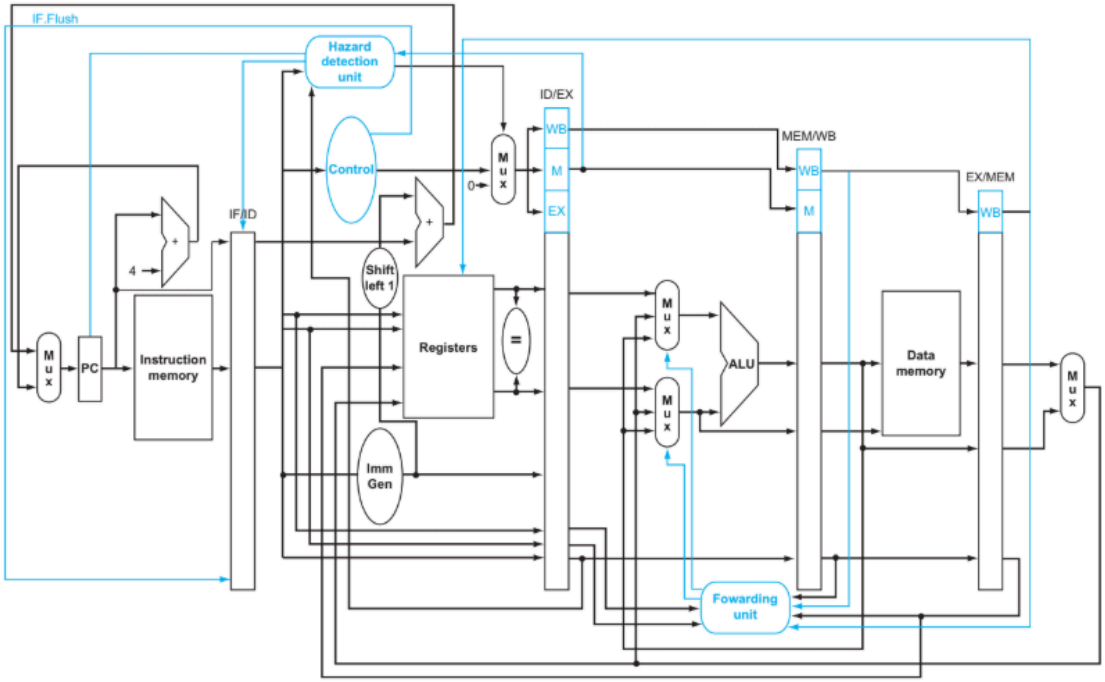
GROUP-2
Code and Module Description:

Top module:

Explanation:

This project has modules for a 32-bit RISC-V processor using a 5-stage pipelined architecture. The five stages are Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM), and Write Back (WB). In the IF stage, the Program Counter (PC) fetches an instruction from instruction memory. In the ID stage, the instruction is decoded, registers are read, and control signals are generated. In the EX stage, the ALU performs arithmetic or logical operations. In the MEM stage, data memory is accessed for



load or store instructions. Finally, in the WB stage, results are written back to the register file. Pipeline registers separate each stage to allow multiple instructions to execute simultaneously, improving performance.

In this top module, signals coming from the IF stage are stored in the IF/ID pipelined register and then mapped to the ID stage, signals coming from the ID stage are stored in the ID/EX pipelined register and then mapped to the EX stage, signals coming from the EX stage are stored in the EX/MEM pipelined register and then mapped to the MEM stage, signals coming from the MEM stage are stored in the MEM/WB pipelined register and then mapped to the WB stage.

In the top module, every module like register file, data memory etc. are instantiated.

PipeLine Flow:

Explanation:

1. **IF_ID_Register**

   The IF_ID_Register is a synchronous 32 bit pipeline register that acts as a buffer between the "Fetch" and "Decode" stages of the processor.

- Inputs:
  - clk: System clock.
  - reset: Synchronous reset signal.
  - ip_instruction [31:0]: The raw instruction fetched from Memory.
  - ip_pc [31:0]: The current Program Counter address associated with that instruction.
- Outputs:
  - op_instruction [31:0]: The buffered instruction passed to the Decode stage.
  - op_pc [31:0]: The buffered PC address passed to the Decode stage.
- Purpose: It "holds" the instruction and its address for exactly one clock cycle. This ensures that while the next instruction is being fetched from memory, the current one remains stable for the Decode Unit to process. It is the fundamental component that enables pipelining, allowing the processor to work on different parts of multiple instructions simultaneously.

2. **ID_EX_Register (Instruction Decode / Execute Stage)**

The **ID_EX_Register** is a synchronous pipeline buffer that separates the **Decode (ID)** and **Execute (EX)** stages. It captures the decoded data and control signals from the instruction and holds them stable for the hardware units (like the ALU and Branch Unit) to use in the next cycle.

Port Declaration

- Inputs:
    - Data: ip_Data_out1, ip_Data_out2 (Register values), ip_immediate (Constants), ip_IF_ID_PC (Program Counter), and ip_Instruction.
    - Control: ip_ALUSrc, ip_ALUOp, ip_MemRead, ip_MemWrite, ip_MemtoReg, ip_RegWrite, and ip_Imm_signal.
    - System: clk (Clock) and reset.
- Outputs:
    - Buffered Signals: Registered versions of all inputs (e.g., op_ID_EX_Data_out1, op_ALUOp, etc.) that update on the rising clock edge.

The primary purpose of this module is to **synchronize the execution stage**. By the time an instruction reaches this register, the processor has already figured out what the instruction is and what data it needs. The **ID_EX_Register** "locks" all this information—the data, the constants, and the control signals—so that the **ALU** and **Branch Unit** can work on them in the next clock cycle without being disturbed by the next instruction coming in behind them.

3. **EX_MEM module** contains all the control signal and the resultant values from the alu after the operation is performed. These values are then passed on to the memory unit where it is decided whether to store the resultant value to the memory or to the specified register. If the value is written in the memory, then all the control signals which activate memory for ex. Memory write data write etc, signals are set.

Port Declaration

- Inputs:
    - Results: ip_ALU (the calculated result), ip_PC (program counter), and

ip_Data_out2 (data to be stored in memory).
    - Control: Signals for the next stages: ip_MemWrite, ip_MemRead, ip_MemtoReg, and ip_RegWrite.
    - System: clk (timed for the negedge in your code) and reset.
- Outputs:
    - Buffered Results: op_EX_MEM_ALU, op_EX_MEM_Data_out2, and op_EX_MEM_PC.
    - Forwarded Control: op_MemWrite, op_MemRead, op_MemtoReg, and op_RegWrite

4. **MEM_WB module**

Inputs:

- **clk:** System clock.
- **reset:** Synchronous reset signal.
- **ip_Read_Data [31:0]:** The data recently loaded from Data Memory.
- **ip_alu_Result [31:0]:** The calculation result from the ALU (used for non-memory instructions).
- **ip_PC [31:0]:** The Program Counter address for the current instruction.
- **ip_MemtoReg [1:0]:** Control signal to select which data to write back to the register.
- **ip_Instruction [31:0]:** The 32-bit machine code of the instruction.
- **ip_RegWrite:** Control signal that enables writing to the Register File.

Outputs:

- **op_Read_Data [31:0]:** The buffered memory data ready for selection.
- **op_alu_Result [31:0]:** The buffered ALU result ready for selection.
- **op_PC [31:0]:** The buffered PC address passed to the final stage.
- **op_MemtoReg [1:0]:** The buffered selection signal for the Write-Back mux.
- **op_Instruction [31:0]:** The buffered instruction used to identify the destination register.
- **op_RegWrite:** The buffered enable signal that finally triggers the register update.

Implements: It "holds" the results of the execution and memory access for exactly one clock cycle to ensure the final Write-Back stage is stable. It captures both memory data and ALU results so the processor can choose the correct value to save back into the

registers without re-fetching or re-calculating. This module completes the pipelining process, ensuring that while one instruction is finishing its write-back, the next one can safely access memory without causing a data conflict.

Hazard detection unit, NOP_mux, Forwarding unit, Forward_4x2_First, Forward_4x2_second:

Explanation:

NOP_mux:
This module is a combinational multiplexer that outputs either a NOP control signal or the normal control signals (concatenated into 10 bits) based on sel_HAZARD_Unit typically for pipeline hazard handling.

FM0:
TheFM0 module is a combinational forwarding multiplexer that selects between ip_IF_ID_RegisterRS1 and ip_EX_MEM_RegisterRS1 based on ip_ForwardC, typically used for data hazard forwarding in a pipelined processor.

FM1:
The Forward_4x2_Second module is a combinational forwarding multiplexer that selects between ip_IF_ID_RegisterRS2 and ip_EX_MEM_RegisterRS2 based on ip_ForwardD, enabling data hazard resolution in a pipelined processor.

Forwarding_Unit Module

The **Forwarding_Unit** module removes **data hazards** in a pipelined processor by forwarding results from later stages (EX/MEM or MEM/WB) to earlier stages without stalling the pipeline.

It checks whether the source registers (RS1, RS2) in the ID/EX stage match the destination registers (RD) in EX/MEM or MEM/WB stages and generates forwarding control signals.

Inputs

- ip_ID_EX_RegisterRS1, ip_ID_EX_RegisterRS2 – Source registers
- ip_EX_MEM_RegisterRD, ip_MEM_WB_RegisterRD – Destination registers
- ip_EX_MEM_RegWrite, ip_MEM_WB_RegWrite – Write control signals

- sel_ALUSrc – ALU source select
- ip_Imm_signal – Immediate indicator

Outputs

- op_ForwardA – Forward control for ALU input A
- op_ForwardB – Forward control for ALU input B
- op_ForwardC, op_ForwardD – Forward control for branch unit.

Hazard_Detection_Unit Module

The **Hazard_Detection_Unit** module detects **load-use data hazards** in a pipelined processor and generates control signals to stall the pipeline when required.

A load-use hazard occurs when an instruction in the ID stage needs data from a previous instruction that is currently performing a memory read in the EX stage. Since the data is not yet available, the pipeline must be stalled.

Inputs

- ip_IF_ID_RegisterRS1 – Source Register 1 from IF/ID stage
- ip_IF_ID_RegisterRS2 – Source Register 2 from IF/ID stage
- ip_ID_EX_RegisterRD – Destination Register from ID/EX stage
- ip_ID_EX_MemRead – Indicates a load instruction in EX stage
- ip_OR_Branch – Branch control signal

Outputs

- op_Hazard_Unit – Hazard detection signal
- ip_PCWrite – Program Counter write control

M0:Reg_Mux_ALU, AU:ALU_Unit, AC:ALU_CTRL_UNIT:

Explanation:

1. ALU_CTRL_UNIT (ALU Control Logic)

This module serves as the primary decoder for the Arithmetic Logic Unit. It translates the general instruction type into a specific 6-bit operation code.

- **Inputs:**
  - Instruction [31:0]: The full 32-bit machine code. The module specifically extracts Instruction[30] and the funct3 field (Instruction[14:12]).
  - ALUOp [1:0]: A signal from the Main Control Unit indicating the instruction category (00 for Load/Store, 10 for R-type, 11 for I-type).
- **Outputs:**
  - ALUCtrl [5:0]: A 6-bit control signal sent directly to the ALU to select the operation.
- **Internal Logic:**
  The module uses nested case statements. If ALUOp is 10 (R-type), it uses both funct3 and Instruction[30] to distinguish between operations like ADD/SUB or SRL/SRA. If ALUOp is 11 (I-type), it performs similar decoding but ignores Instruction[30] for certain immediate operations. For ALUOp 00 (Load/Store), it bypasses detailed decoding and outputs the add_ctrl signal to calculate memory addresses.

2. ALU_Unit (Arithmetic and Logic Execution)

This module is the functional block that performs the mathematical or logical computation on the data.

- **Inputs:**
  - Read_data1 [31:0]: The first operand, typically sourced from the first register (rs1).
  - Mux_Data [31:0]: The second operand, sourced from either the second register (rs2) or an immediate value via a multiplexer.
  - ALU_Control [5:0]: The 6-bit command generated by the ALU_CTRL_UNIT.
- **Outputs:**

    - ALU_Result [31:0]: The 32-bit numerical or logical result of the operation.
- **Internal Logic:**
  The module implements a combinational always @(*) block with a case statement driven by ALU_Control.
    - **Arithmetic:** Performs Read_data1 + Mux_Data or Read_data1 - Mux_Data.
    - **Bitwise:** Executes AND, OR, and XOR operations.
    - **Shifting:** Supports logical left/right shifts and arithmetic right shifts (using >>> to maintain the sign bit).
    - **Comparisons:** For slt (Set Less Than), it performs a signed comparison; for sltu, it performs an unsigned comparison, returning 1 if the condition is true and 0 otherwise.

3. Reg_Mux_ALU (Datapath Integration)

This module acts as a data selector (multiplexer) that decides which value should be fed into the second input of the ALU. It ensures the processor can handle both register-to-register and register-to-constant operations.

- **Inputs:**
    - Read_data2 [31:0]: The 32-bit value retrieved from the second source register (rs2) in the Register File.
    - Immediate_Gen [31:0]: A 32-bit constant value extracted and sign-extended from the instruction (used for "Immediate" instructions like addi or lw).
    - ALUSrc: A 1-bit control signal from the Main Control Unit that acts as the selector switch.
- **Outputs:**
    - Mux_Data [31:0]: The chosen 32-bit operand that is passed directly into the second input port of the ALU_Unit.
- **How it works:**
  The module operates as a simple gatekeeper. When the ALUSrc signal is low (0), it passes the value from the register (Read_data2). When the ALUSrc signal is high (1), it bypasses the register and passes the Immediate_Gen value instead. This mechanism is what allows a single ALU to perform different types of instructions, such as adding two variables together or adding a specific number to a variable.

CU:Control_Unit, Reg:Reg_file, Imm:Immediate

Explanation:

Control Unit (CU):

The Control Unit is a combinational instruction-decoding module that acts as the central control logic of the processor, translating the 32-bit instruction into a set of coordinated control signals for the entire datapath.

Inputs: Instruction [31:0] – the raw instruction fetched from instruction memory.

Outputs: RegWrite, ALUSrc [1:0], ALUOp [1:0], MemtoReg [1:0], MemRead, MemWrite, Branch_En, Jump, Imm_signal – control signals that drive the Register File, ALU, Immediate Generator, Data Memory, Writeback MUX, and PC control logic.

Purpose: It decodes the opcode field of the instruction and generates the appropriate control configuration that determines operand selection, ALU operation, memory access, writeback routing, and control-flow behavior. This module ensures that each instruction is executed correctly by dynamically configuring the hardware resources of the processor for arithmetic, memory, and control-flow operations.

RegisterFile(RegFile):

The Reg_file module is a synchronous 32×32-bit multi-port register storage unit that serves as the primary operand storage and distribution block of the processor.

Inputs: clk – system clock, reset – synchronous reset signal, Data_in [31:0] – writeback data from the datapath, Read_Reg1 [4:0], Read_Reg2 [4:0] – source register addresses (rs1, rs2), Write_Reg [4:0] – destination register address (rd), RegWrite – write enable control signal.

Outputs: Data_out1 [31:0], Data_out2 [31:0] – operand values supplied to the execution stage.

Purpose: It provides two simultaneous read ports and one write port, allowing the processor to fetch two operands and write one result in a single cycle. The module

includes reset initialization, architectural protection of register x0, and internal bypass logic to resolve same-cycle read-after-write hazards. It enables high-speed operand access, pipeline concurrency, and correct instruction execution by acting as the central storage hub for all computational data within the CPU.

Immediate Module:

The **Immediate** module is a combinational 32-bit immediate generation unit that acts as the constant extraction and formatting block of the processor, responsible for constructing immediate values from RISC-V instruction encodings.

Inputs: instruction [31:0] – the raw instruction fetched from instruction memory.

Outputs: immediate [31:0] – the sign-extended 32-bit immediate value supplied to the ALU and PC logic.

Purpose: It decodes the opcode field to identify the instruction format (I, S, B, U, J types), extracts the scattered immediate bit-fields from the instruction, reconstructs them into the correct format, and performs sign-extension to produce a uniform 32-bit value. This ensures correct execution of arithmetic operations, address calculations, branching, and jumping by providing a consistent and architecture-compliant immediate representation to the datapath.

Add_J, Branch Unit:

Explanation:

Module Branch Unit:

This module implements the branch decision logic for B-type instructions in the RISC-V ISA.
It takes the 32-bit Instruction and extracts the funct3 field from bits [14:12], which determines the specific branch type. Branch_En acts as a control signal; the branch condition is evaluated only when this signal is 1. The module compares ip_read_data1 (rs1) and ip_read_data2 (rs2) based on funct3:
000, 001,---, 111 denote different branch instructions like bne, beq, etc.
Signed comparisons use $signed, while unsigned comparisons use $unsigned to match RISC-V ISA behaviour. We have used case statements to implement the different branch instructions mentioned above. The output op_Branch_Unit becomes 1 if the branch condition is true (branch taken), otherwise 0 (branch not taken).

Module Add_J

The Add_J module implements a 32-bit combinational adder used for jump address calculation
It has two 32-bit inputs: ip_Add_PC and ip_Shift.
The output op_Add is a 32-bit register that stores the result of the addition.

IF_WRAPPER, DATA_MEMORY_WRAPPER:

1. IF_WRAPPER

   This module implements the Instruction Fetch (IF) stage. It takes the PC address and reads the corresponding 32-bit instruction from block RAM. The fetched instruction is provided as output op_IF.
   Inputs

- **ip_PC_Add [7:0]:** 8-bit Program Counter address.
- **clk:** Clock input used to synchronize instruction memory access

  Output

- **op_IF [31:0]**: 32-bit instruction fetched from instruction memory (IM).

2. DATA_MEMORY_WRAPPER
   Inputs
   - **BRAM_PORTA_0_addr [9:0]:** Write address (Port A) of data memory.
   - **i_clk:** Clock signal for synchronous BRAM access.
   - **BRAM_PORTA_0_din [31:0]**: 32-bit data to be written into memory during store operation.
   - **BRAM_PORTB_0_addr [9:0]** :Read address (Port B) of data memory.
   - **MemWrite**: Enables store operation.
   - **MemRead**: Enables load operation.
   - **funct3 [2:0]**: Determines load/store type (byte, half-word, word, signed/unsigned).

   Output
   - **BRAM_PORTB_Wr_out [31:0]**: Loaded data after proper sign/zero extension.

   This module implements the Data Memory stage with load-store control.

- The funct3 field decides whether the memory operation works on a byte (8-bit), half-word (16-bit), or full word (32-bit).

- If MemWrite = 1, the module enables the correct byte lanes and writes the selected portion of data into memory on the next clock edge.

- If MemRead = 1, it reads data from memory and selects the required byte, half-word, or word.

- For signed loads (LB, LH), the value is sign-extended to 32 bits, while for unsigned loads (LBU, LHU), the upper bits are filled with zeros.

ADD_PC, Mux_PC:

ADD_PC:-

Inputs

- **ip_clk:** Clock signal for synchronous PC update (positive-edge triggered).

- **ip_rst:** Reset signal. When asserted, PC is forced to 0.

- **ip_add [31:0]:** Current Program Counter (PC) value.

- **ip_PCWrite:** Control signal from Hazard Detection Unit.
  Determines whether normal PC increment or redirected PC update occurs.

- **ip_delay_PC [31:0]:** Alternate PC value (typically branch/jump target).

Output

- **op_add [31:0]:** Updated Program Counter value.

Explanation: The Add_PC module in a RISC-V processor is used to calculate the next value of the Program Counter (PC). It works on the positive edge of the clock and updates the PC based on different control signals. Its operation can be summarized in three main cases:

1. When the reset signal is high, the PC is set to 0 so that execution starts from the

beginning.

2. When reset is low and ip_PCWrite = 0, the PC increases by 4 (PC + 4) to move to the next sequential instruction.

3. When ip_PCWrite = 1, the PC is updated with ip_delay_PC, which is typically used during branch, jump, or hazard conditions to change the normal flow of execution.

Mux_PC:-

Inputs

- **ip_Add_PC [31:0]:** Sequential PC address (typically PC+4PC + 4PC+4).

- **ip_Add_J [31:0]:** Branch or jump target address.

- **sel_PCSrc:** Control signal selecting the next PC source.

Output

- **op_Mux_pc [31:0]:** Selected next Program Counter value

Explanation: The Mux_PC module in a RISC-V processor is used to select the next value of the Program Counter (PC) using a control signal. It works as a combinational multiplexer that chooses between the normal sequential address and a jump/branch address. Its operation can be summarized in three cases:

1. When sel_PCSrc = 0, the output is ip_Add_PC, meaning the processor continues normal execution (PC + 4).

2. When sel_PCSrc = 1, the output is ip_Add_J, meaning the processor jumps to a branch or jump target address.

3. If the select signal is invalid (default case), the output is set to 0 as a safety condition.